

FORMAL AGENT NOTATION

Extended Draft 1.1 · Multi-Agent Orchestration Framework

Original: Draft 1.0 · Extensions marked [NEW] or [MODIFIED]

1. Core State & Environment Definitions

Unchanged from Draft 1.0. Reproduced for reference.

I – space of all inputs
O – space of all outputs
S – internal agent state space (context window / scratchpad)
E – environment state topological space
T – set of tools $\tau: O_{call} \times E \rightarrow I_{result} \times E_{mutated}$

2. The Formal Agent A

A = $\langle I, O, S, T, \pi_{\theta}, \delta \rangle$
 $\pi_{\theta}(o \mid i, s)$ – stochastic policy over outputs
 $\delta: S \times I \times O \rightarrow S$ – state transition / memory update

2.1 Execution Step

$o_t \sim \pi_{\theta}(\cdot \mid i_t, s_t)$
 $s_{t+1} = \delta(s_t, i_t, o_t)$
Shorthand: $A(i, s) \rightarrow (o, s')$

[NEW]

2.2 Shared Environment **E_shared**

The original formulation treats **E** as local to each agent. For multi-agent systems a single environment instance must be shared and its access made explicit.

E_shared = (E, Ω, R_{access})
E – the shared environment state (e.g. a git repository, SQLite DB)
Omega – a set of access operations: { read, write, lock, unlock, cas }
R_access – an access-control relation: $R_{access} \subset (A \times \Omega)$

A tool operating on the shared environment is now typed as:

$\tau_{shared}: O_{call} \times E_{shared} \rightarrow I_{result} \times E_{shared}$

Crucially, writes to **E_shared** are visible to all agents whose **R_access** permits reads. This makes **E_shared** the primary coordination channel in systems without direct inter-agent messaging.

[NEW]

2.3 Communication Channel Ch

A Channel is a typed, persistent coordination artifact that lives inside E_{shared} . Unlike direct I/O piping between agents, a channel decouples writers from readers in time — critical for turn-based systems with no live messaging.

Ch = (K, V, order, Phi_access)

K – key space (e.g. section name, agent ID, task ID)

V – value space (structured payloads: JSON, Markdown, etc.)

order – FIFO | LIFO | priority | unordered

Phi_access – predicate controlling who may read/write each key

Channel operations:

Ch.write(k, v, A_i) – agent A_i writes value v under key k

Ch.read(k, A_j) – agent A_j reads value at key k

Ch.claim(k, A_i) – atomic ownership claim (CAS semantics)

Ch.release(k, A_i) – release ownership

Examples of concrete channels:

Ch_git – a git repository ($K=\text{branch/path}$, $V=\text{file content+commits}$)

Ch_file – a single shared file ($K=\text{section_name}$, $V=\text{text block}$)

Ch_db – SQLite message table ($K=\text{row_id}$, $V=\text{JSON payload}$)

3. Multi-Agent Composition Operators

Operators 3.1 – 3.6 are unchanged from Draft 1.0. New operators 3.7 – 3.9 follow.

3.1 Sequential (→)

$A \rightarrow B \text{ equiv } B(A(i)_{\text{out}}, s_B)$

3.2 Parallel / Broadcast (||)

$||_{k=1}^n A_k(i) \rightarrow [A_1(i), A_2(i), \dots, A_n(i)]$

3.3 Conditional Routing (|→)

$R | \rightarrow \{ A_1, \dots, A_n \}$ resolves to A_k iff $R_{\text{cls}}(i) = k$

3.4 Synthesizer / Map-Reduce (<→)

$S \leftrightarrow (W_1 || W_2 || \dots || W_n)$

3.5 Reflection / Iterative Loop (↶)

$A^{\{\text{loop } \Phi\}}$ iterates until $\Phi_{\text{stop}}(o, s) = \text{True}$

3.6 Hierarchical Delegation (>)

$H > W$ H retains interrupt rights; W is a bounded subroutine

[NEW]

3.7 Synchronisation Barrier (`|| _sync`)

A barrier forces a set of agents to reach a defined checkpoint state before any of them may proceed. Essential for turn-based systems where Agent B must not start until Agent A has committed its output.

Barrier(A_1, ..., A_n | Phi_ready)

Phi_ready(A_i, Ch) – predicate evaluated per agent against channel state

All agents block until forall i: Phi_ready(A_i, Ch) = True

Shorthand in composition expressions:

A_1 -> [[Phi_ready]] -> A_2

A_2 does not begin until Phi_ready holds in Ch (e.g. TURN=A_2 in shared file)

[NEW]

3.8 Peer Review Gate (`~>`)

A Peer Review Gate is an asymmetric composition: Agent B reads Agent A's committed output in Ch and produces a review verdict before the output is allowed to propagate downstream. Unlike delegation ($H > W$), neither agent has authority over the other — the gate is horizontal.

A ~> B (B reviews A's output in Ch)

A writes o_A to Ch

B reads o_A, produces verdict v in { approve, reject, flag }

If v = approve: o_A propagates; if reject: routed back to A or escalated

Extended form with explicit channel and verdict routing:

A ~>[Ch, Phi_accept] B -> A_downstream

Phi_accept(v) controls whether output passes the gate

[NEW]

3.9 Contract Negotiation & Lock (`<=>`)

Used when two or more agents must agree on a shared interface specification before any execution proceeds. Each agent proposes; all must ratify; the resulting contract Gamma is immutable for the remainder of the session unless explicitly unlocked by a supervisor.

(A_1 <=> A_2)[Ch_plan] -> lock(Gamma)

A_1 writes proposal p_1 to Ch_plan

A_2 reads p_1, writes counter-proposal or ratification to Ch_plan

Gamma = merge(p_1, p_2) once all parties have written "confirmed"

Ch_plan is then marked LOCKED – no further writes permitted

Post-lock, both agents execute against the contract:

A_1(Gamma) || A_2(Gamma)

Each agent receives Gamma as a constraint appended to its input context

VERSION 1 · Independent Work + Peer Review

Expressed in Formal Agent Notation 1.1

Agent Definitions

A_1 = < I, 0, S, T_git, pi_theta, delta > // your Claude Code instance

A_2 = < I, 0, S, T_git, pi_theta, delta > // friend's Claude Code instance

Ch_git = git repository (K=branch/path, V=commits, order=DAG)

```
E_shared = (Ch_git, { read, write, branch, merge, pr }, R_access)
```

R_access: both A_1 and A_2 have full read; write restricted to own branch

Phase 1 · Orientation (both agents, T=0)

```
A_1(fetch(E_shared), s_1) -> (branch_map, s_1')
```

```
A_2(fetch(E_shared), s_2) -> (branch_map, s_2')
```

Each agent reads the full branch namespace from Ch_git.

Branch names act as ownership claims – no explicit lock protocol needed.

Both agents execute this step in parallel; no dependency between them.

```
Formally: || [ A_1.fetch(Ch_git), A_2.fetch(Ch_git) ]
```

Phase 2 · Parallel Execution (T=1 to T=8)

```
A_1 : Ch_git.claim("agent-1/auth-system")
```

```
A_2 : Ch_git.claim("agent-2/db-schema")
```

```
|| [
```

```
A_1^{loop Phi_1}(task_auth, s_1) -> (o_auth, s_1')
```

```
A_2^{loop Phi_2}(task_schema, s_2) -> (o_schema, s_2')
```

```
]
```

Phi_1 = "all auth tests passing, PR ready"

Phi_2 = "schema migrated, models committed, PR ready"

Each loop iteration contains an implicit periodic fetch:

```
A_i step k: o_k, s_{k+1} <- A_i(i_k, s_k)
```

```
i_{k+1} = fetch(Ch_git) + task_context
```

The fetch surfaces the other agent's commits as passive read-only context.

No blocking. Each agent continues its own loop regardless.

Phase 3 · Peer Review Gate (T=8 to T=11)

When A_2 opens its PR, A_1 performs a peer review before the merge proceeds. This uses the new ~> operator from §3.8.

```
A_2 ~>[Ch_git, Phi_pr_accept] A_1 -> merge(Ch_git, "main")
```

A_2 pushes branch and opens PR (write to Ch_git)

A_1 reads PR diff from Ch_git

A_1 produces verdict v in { approve, request_changes }

Phi_pr_accept(v) = True iff v = approve

On approval: PR merges into main; A_2's branch claim released

Symmetrically, A_2 reviews A_1's PR:

```
A_1 ~>[Ch_git, Phi_pr_accept] A_2 -> merge(Ch_git, "main")
```

Full phase 3 expression (reviews can be concurrent if both PRs are open):

```
|| [
```

```
A_2 ~>[Ch_git, Phi_pr_accept] A_1,
```

```
A_1 ~>[Ch_git, Phi_pr_accept] A_2
]
```

Each review is independent; both merges proceed once approved.

Complete Version 1 Expression

```
V1(task) =
```

```
/* Phase 1: Orientation */
|[ [ A_1.fetch(Ch_git), A_2.fetch(Ch_git) ]

/* Phase 2: Parallel execution in claimed branches */
-> |[
A_1^{loop Phi_1}( Ch_git.claim("agent-1/auth"), task_auth ),
A_2^{loop Phi_2}( Ch_git.claim("agent-2/schema"), task_schema )
]

/* Phase 3: Cross peer-review then merge */
-> |[
A_2 ~>[Ch_git, Phi_accept] A_1,
A_1 ~>[Ch_git, Phi_accept] A_2
]

-> merge(Ch_git, "main")
```

Reading: orientation in parallel, then parallel execution each in own loop, then cross-review in parallel, then integration. The -> separates phases (each phase begins only when the prior is resolved), || within a phase expresses genuine concurrency.

Property	Value
Coupling	Loose — agents share Ch_git reads; no direct message passing
Coordination	Passive — branch namespace + commit messages carry intent
Blocking	None during execution; only at peer-review gate
Conflict detect	At merge time by Ch_git (git); not at write time
Failure model	Stale branch claim detected via heartbeat timestamp in Ch_git